LAPORAN PRAKTIKUM SESI 10

KOMUNIKASI ANTARSERVICE VIA MESSAGE QUEUE

Penerapan RabbitMQ pada Marketplace UMKM

Mata Kuliah Web Service



Dosen Pengampu

Philipus Suryo Subandoro

Disusun oleh

Bambang Herlambang 5803024019

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS KATOLIK WIDYA MANDALA SURABAYA

2026

DAFTAR ISI

BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan Praktikum 1
1.3 Ruang Lingkup 1
BAB II DASAR TEORI 2
2.1 Komunikasi Sinkron dan Asinkron 2
2.2 RabbitMQ dan Exchange Fanout 2
2.3 Acknowledgement dan Idempotensi 2
BAB III IMPLEMENTASI SISTEM 3
3.1 Arsitektur dan Kepemilikan Data 3
3.2 Kontrak Event dan Penanganan Pesan 4
3.3 Lingkungan dan Prosedur Menjalankan 4
BAB IV HASIL PENGUJIAN DAN PEMBAHASAN 5
4.1 Sumber Bukti Pengujian 5
4.2 Consumer Mati dan Pesan Menunggu 5
4.3 Consumer Dinyalakan Kembali 6
4.4 Crash Sebelum Ack dan Redelivery 7
4.5 Competing Consumers 8
4.6 Publish Subscribe 9
4.7 Pengujian Tambahan dan Konsistensi 10
BAB V PENUTUP 11
5.1 Penerapan pada Proyek Marketplace UMKM 11
5.2 Refleksi 11
5.3 Kesimpulan dan Batasan 11
DAFTAR PUSTAKA 12
LAMPIRAN SUMBER DAN REPRODUKSI 13
LAMPIRAN BUKTI REQUEST POSTMAN 14

BAB I
PENDAHULUAN

1.1 Latar Belakang

Praktikum sesi 10 melanjutkan pemisahan aplikasi Marketplace UMKM pada Lab 09 dengan menambahkan komunikasi asinkron melalui RabbitMQ. Reservasi stok tetap dilakukan melalui HTTP karena hasilnya dibutuhkan sebelum pesanan diterima. Pekerjaan notifikasi dipindahkan ke consumer terpisah agar pesanan tidak perlu menunggu pengiriman email.

Hasil pengujian menunjukkan bahwa pesanan tetap diterima ketika consumer berhenti, sedangkan pesan menunggu di antrean dan diproses setelah consumer pulih. Pengujian crash juga menunjukkan bahwa pesan dapat diproses dua kali apabila consumer berhenti sebelum memberikan acknowledgement.

1.2 Tujuan Praktikum

Praktikum bertujuan menjalankan RabbitMQ dan mengamati status antreannya, membangun producer dan consumer terpisah, membuktikan penahanan pesan saat consumer mati, serta menjelaskan acknowledgement, redelivery, competing consumers, pub/sub, dan kebutuhan idempotensi.

1.3 Ruang Lingkup

Implementasi mencakup gateway, Catalog Service, Order Service, RabbitMQ, dan Notification Service. Pesanan serta stok tetap disimpan dalam memori. Email yang ditampilkan merupakan simulasi log; aplikasi belum terhubung ke SMTP, database persisten, atau transactional outbox.

Bukti utama butir consumer mati dan crash berasal dari pengujian manual pada 4 November 2026. Pengujian otomatis melengkapi pembuktian competing consumers, pub/sub, serta penanganan kegagalan. Keduanya menggunakan ID, waktu, dan virtual host yang berbeda.

BAB II
DASAR TEORI

2.1 Komunikasi Sinkron dan Asinkron

Pada komunikasi sinkron, pemanggil menunggu respons service tujuan. Pola ini sesuai untuk pemeriksaan stok dan harga saat checkout. Pada komunikasi asinkron, producer menerbitkan event ke broker dan consumer memprosesnya kemudian. Balasan HTTP tidak menunggu pekerjaan consumer selesai.

2.2 RabbitMQ dan Exchange Fanout

RabbitMQ menyalurkan pesan melalui exchange ke antrean yang terikat. Exchange fanout mengirim salinan pesan ke setiap antrean terikat. Pada praktikum ini, exchange orders menghubungkan notification-queue dan, pada eksperimen pub/sub, audit-queue. Dua consumer pada antrean yang sama berbagi pekerjaan, sedangkan consumer pada antrean berbeda menerima salinan masing-masing.

Exchange dan antrean menggunakan durable=true; pesan memakai persistent=true. Volume Docker mempertahankan data broker. Konfigurasi ini melindungi pesan yang sudah diterima broker, tetapi tidak menyatukan penyimpanan pesanan dan publikasi event menjadi satu transaksi. Exchange sendiri tidak menahan event tanpa antrean tujuan.

2.3 Acknowledgement dan Idempotensi

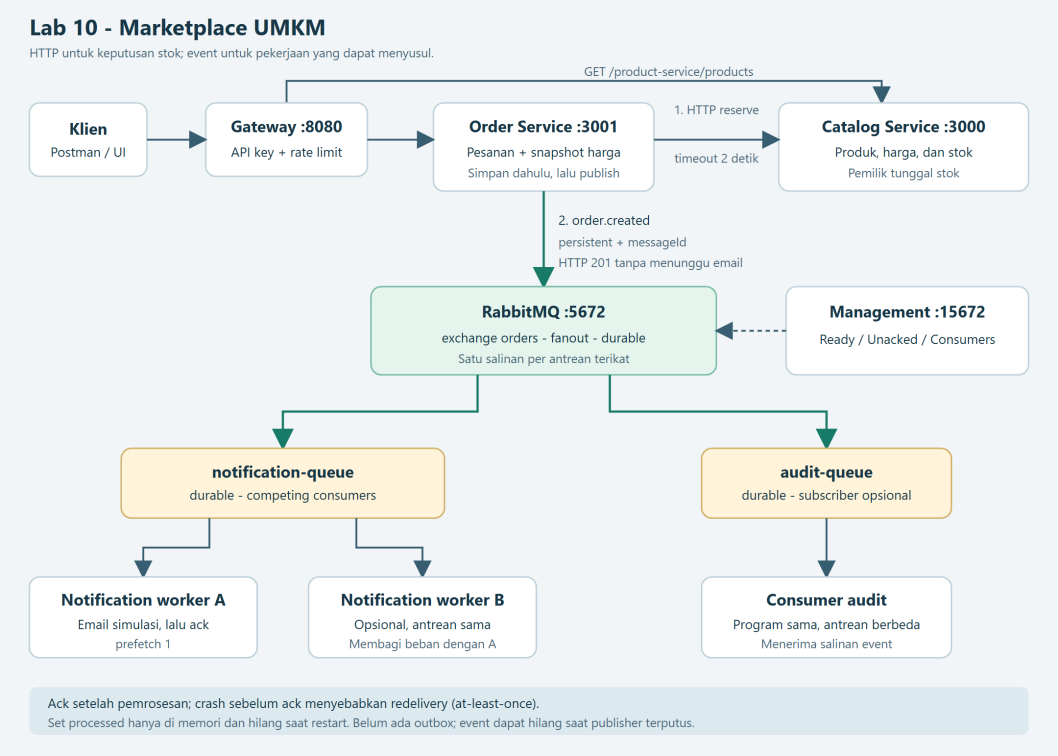
Consumer memberikan ack setelah pekerjaan selesai. Jika koneksi terputus sebelum ack, broker dapat mengirim ulang pesan yang belum diakui. Inilah perilaku at-least-once pada pesan yang telah diantrekan, bukan jaminan side effect terjadi tepat sekali.

Consumer idempotent harus mencegah side effect berulang untuk messageId yang sama. Set dalam memori hanya mengenali duplikat selama proses hidup. Penyimpanan tahan restart dengan unique constraint dan transaksi diperlukan untuk side effect database; email eksternal memerlukan dukungan idempotency key dari penyedia atau rancangan pengiriman yang sesuai.

BAB III
IMPLEMENTASI SISTEM

3.1 Arsitektur dan Kepemilikan Data

Gateway meneruskan request ke katalog atau pesanan. Catalog Service menjadi pemilik tunggal stok dan harga. Order Service meminta reservasi melalui HTTP dengan timeout dua detik, menyimpan pesanan beserta snapshot harga, lalu menerbitkan order.created. Notification Service menerima pesan melalui RabbitMQ dan memberikan ack setelah email simulasi dicetak.



Gambar 3.1 Arsitektur Lab 10 dengan alur reservasi sinkron dan notifikasi asinkron.

Komponen	Data dan tanggung jawab
Catalog Service :3000	Produk, harga, dan stok; satu-satunya pengubah stok.
Order Service :3001	Pesanan dan snapshot harga; producer order.created.
RabbitMQ :5672	Exchange orders dan antrean durable; dashboard :15672.
Notification Service	Consumer, email simulasi, Set messageId, manual ack, prefetch 1.

3.2 Kontrak Event dan Penanganan Pesan

Payload memuat type=order.created dan data pesanan: id, customerName, items, total, status, serta createdAt. Setiap item berisi productId, name, quantity, unitPrice, dan subtotal. AMQP properties menggunakan messageId=order-${order.id}, contentType=application/json, dan persistent=true.

Publisher menggunakan confirm channel secara asinkron dan mandatory=true. Confirm menunjukkan penerimaan publikasi oleh broker, bukan selesainya email. Event tanpa antrean tujuan dicatat sebagai unroutable. Nilai false dari channel.publish berarti buffer menahan backpressure, bukan pesan saat itu ditolak.

Publisher mencoba reconnect setiap tiga detik ketika broker tidak tersedia. Order tetap dapat menerima HTTP 201, tetapi event yang tidak terkirim tidak direplay otomatis. Consumer melakukan manual ack; event atau JSON tidak valid di-nack tanpa requeue. Consumer perlu dijalankan kembali jika koneksi brokernya terputus.

3.3 Lingkungan dan Prosedur Menjalankan

Lingkungan lokal menggunakan Node.js 22 atau lebih baru dan Docker Desktop dengan Linux engine. Persiapan dari root repository adalah npm ci, npm run setup, dan npm run broker:up. Consumer dijalankan dengan npm run start:notification sebelum order pertama diterbitkan agar antrean dan binding sudah terbentuk.

Service HTTP dijalankan melalui npm start. Setelah publisher mencetak ready, request dapat dikirim melalui Postman atau npm run test:smoke. Gateway memakai alamat http://127.0.0.1:8080 dan header X-API-Key dari konfigurasi lokal. Dashboard RabbitMQ tersedia di http://127.0.0.1:15672. Kredensial cloud hanya disimpan melalui environment AMQP_URL.

Pengujian otomatis memakai npm test dan npm run test:lab10:evidence. Demo memilih port HTTP bebas dan vhost baru agar tidak mengubah antrean pengguna. Screenshot otomatis memerlukan instalasi browser melalui npx playwright install chromium.

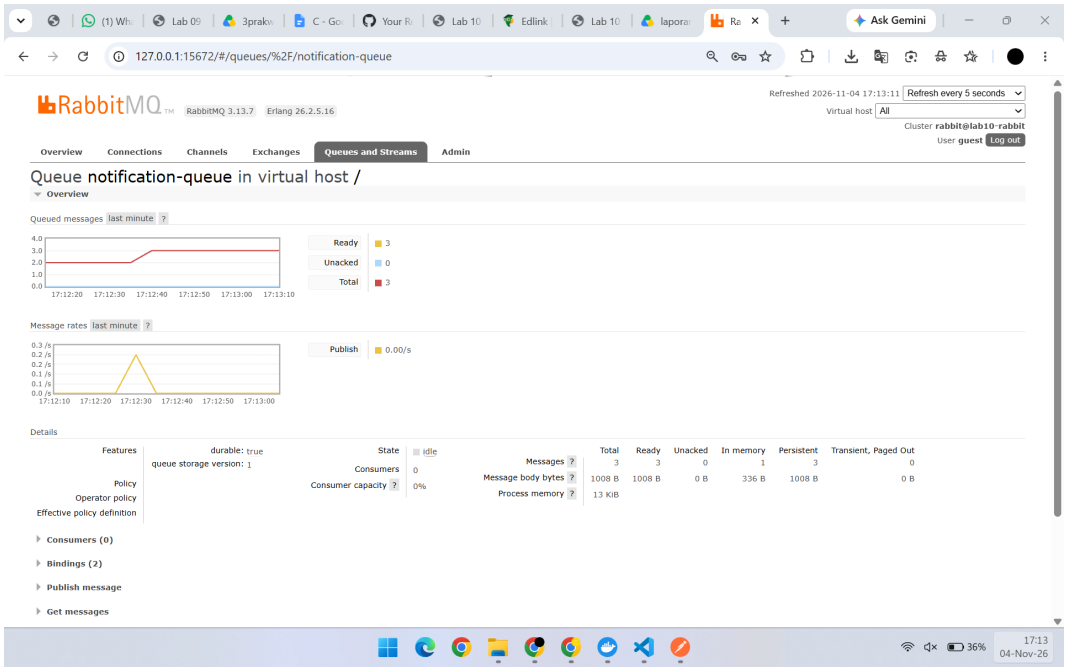
BAB IV
HASIL PENGUJIAN DAN PEMBAHASAN

4.1 Sumber Bukti Pengujian

Pengujian manual menggunakan Postman, RabbitMQ Management UI, dan terminal VS Code pada virtual host /. Screenshot disalin utuh ke repository. Tabel waktu berikut merupakan transkripsi tampilan, dengan timestamp UTC pada log dikonversi ke UTC+7. Run otomatis sebelumnya memakai vhost terisolasi dan diperlakukan sebagai sumber bukti tersendiri.

4.2 Consumer Mati dan Pesan Menunggu

Setelah consumer dihentikan, tiga request pada folder Postman 02 Consumer mati masing-masing mendapat 201 Created. Ketiga event menunggu di notification-queue ketika jumlah consumer nol. Pengirim dan penerima tidak harus aktif bersamaan jika broker aktif dan antrean sudah terikat.

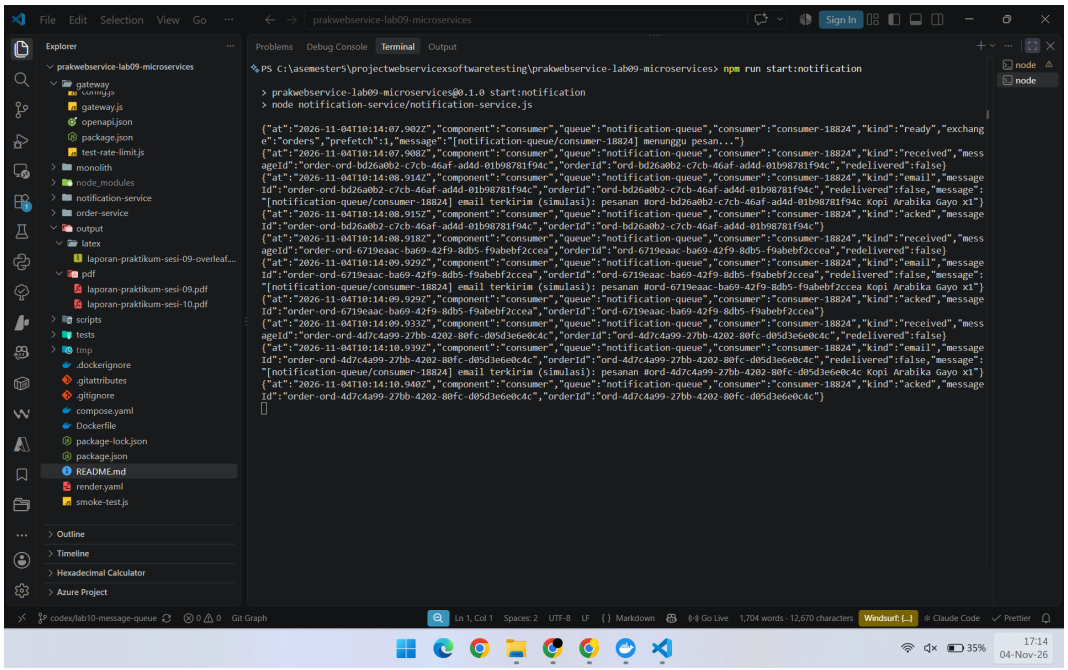


Gambar 4.1 Dashboard manual pada 4 November 2026 pukul 17:13:11 UTC+7. Judul halaman menunjukkan notification-queue pada vhost /.

Pengamatan	Hasil manual
Ready / Unacked / Consumers	3 / 0 / 0
Respons tiga request Postman	Masing-masing 201 Created

4.3 Consumer Dinyalakan Kembali

Consumer dijalankan kembali melalui npm run start:notification. Consumer-18824 memproses ketiga pesan tertahan. Setiap email simulasi diikuti ack dan orderId-nya cocok dengan respons Postman sebelumnya.



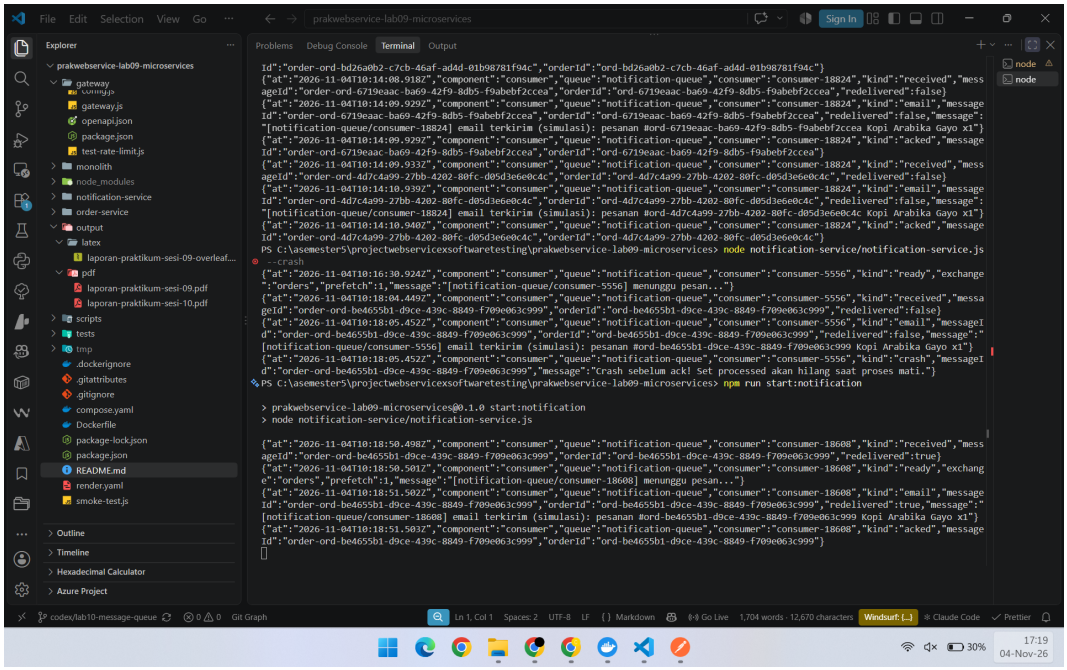
Gambar 4.2 Terminal manual setelah consumer kembali berjalan dan memproses tiga pesan.

OrderId sesuai Postman	Email / ack UTC+7
ord-bd26a0b2-c7cb-46af-ad4d-01b98781f94c	17:14:08.914 / 17:14:08.915
ord-6719eaac-ba69-42f9-8db5-f9abebf2ccea	17:14:09.929 / 17:14:09.929
ord-4d7c4a99-27bb-4202-80fc-d05d3e6e0c4c	17:14:10.939 / 17:14:10.940

Bukti manual pemulihan memperlihatkan tiga email dan tiga acknowledgement. Hal ini melengkapi screenshot Ready=3 sebelumnya dan membuktikan bahwa pesan yang tertahan dapat diproses setelah consumer pulih.

4.4 Crash Sebelum Ack dan Redelivery

Consumer dimulai dengan --crash, lalu satu request pada folder Postman 05 Crash dikirim dan mendapat 201 Created. Consumer-5556 mencetak email simulasi, menampilkan Crash sebelum ack!, lalu berhenti. Tanpa mengirim request baru, consumer biasa dijalankan kembali dan memproses pesan yang sama.



Gambar 4.3 Terminal manual menunjukkan email pertama sebelum crash dan email kedua sesudah restart.

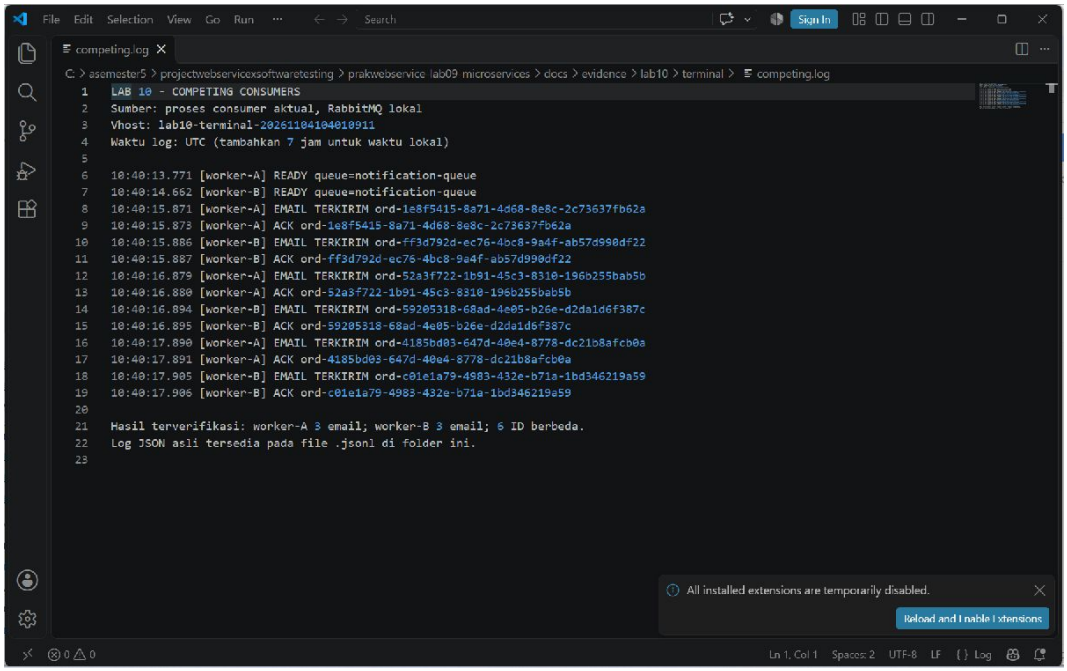
MessageId kedua pemrosesan adalah order-ord-be4655b1-d9ce-439c-8849-f709e063c999.

Tahap	Waktu UTC+7	Hasil
Email pertama	17:18:05.452	consumer-5556; redelivered:false; crash sebelum ack
Email kedua	17:18:51.502	consumer-18608; redelivered:true
Ack	17:18:51.503	consumer-18608; kind: acked

Email tercetak dua kali untuk messageId yang sama karena broker belum menerima ack pertama dan Set processed hilang saat restart. Hasil ini menunjukkan perlunya idempotensi tahan restart.

4.5 Competing Consumers

Pengujian tambahan pada 4 November 2026 pukul 17:40 UTC+7 menjalankan dua consumer pada notification-queue yang sama. Enam order dikirim melalui gateway dan seluruhnya mendapat HTTP 201. Event dibagi antara worker-A dan worker-B, tanpa ID yang diproses oleh kedua worker.



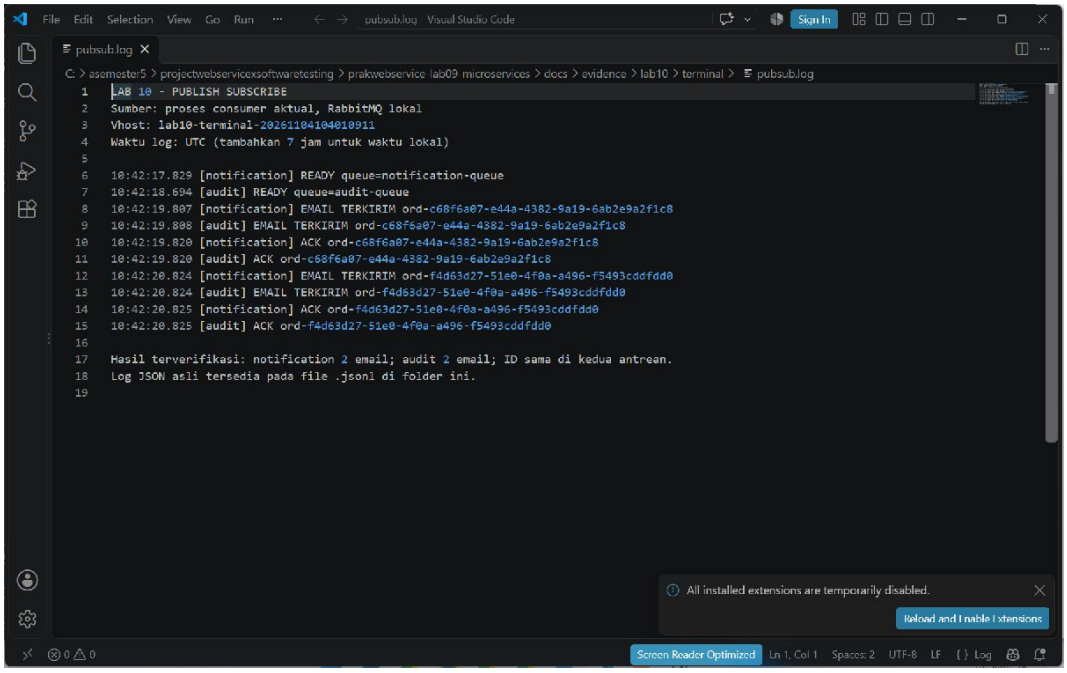
Gambar 4.4 Screenshot VS Code yang menampilkan log consumer aktual. Tampilan ringkas mempertahankan timestamp, nama worker, jenis event, dan orderId dari file JSONL asli; bukan halaman hasil HTML.

Consumer	Email simulasi	Acknowledgement
worker-A	3	3
worker-B	3	3

Prefetch 1 membatasi satu pesan unacked per worker. Pembagian 3+3 adalah hasil run ini, bukan jaminan pembagian selalu seimbang. Bukti ini memakai virtual host tambahan yang terpisah dari antrean manual pengguna.

4.6 Publish Subscribe

Pada pukul 17:42 UTC+7, satu consumer membaca notification-queue dan satu consumer lain membaca audit-queue. Kedua antrean terikat ke exchange orders. Dua order baru dikirim; setiap antrean menerima salinan kedua event dengan orderId yang sama.



Gambar 4.5 Screenshot VS Code atas log consumer aktual pada dua antrean. Setiap orderId terlihat pada consumer notification dan audit, masing-masing diikuti ack.

OrderId yang diterbitkan	Penerima
ord-c68f6a07-e44a-4382-9a19-6ab2e9a2f1c8	notification dan audit
ord-f4d63d27-51e0-4f0a-a496-f5493cddfdd0	notification dan audit

Fanout mengirim satu salinan ke setiap antrean terikat. Pola ini berbeda dari pembagian pekerjaan pada satu antrean. Consumer audit memakai program simulasi yang sama dengan nama antrean berbeda; belum ada database audit produksi.

4.7 Pengujian Tambahan dan Konsistensi

Pengujian	Hasil
Alur normal	HTTP 201 diterima sebelum email simulasi selesai.
Duplikat pada proses sama	ID lama di-ack dan dilewati tanpa email baru.
Pesan rusak	JSON tidak valid di-nack tanpa requeue.
Publisher terputus	Order tetap 201; event tidak terkirim. Setelah reconnect, event order baru terkirim.
Suite Lab 09 dan Lab 10	75 dari 75 test lulus; 0 gagal.

Skenario gangguan publisher menutup hanya koneksi AMQP publisher melalui Management API, bukan seluruh broker. Unit test juga memeriksa kegagalan koneksi awal, channel error, backpressure, dan ack/nack. Delapan skenario integrasi RabbitMQ asli lulus pada run otomatis.

Penyimpanan order dan publikasi event belum atomik. Jika RabbitMQ tidak tersedia, order dapat tersimpan tanpa notifikasi. Durable queue, persistent message, dan publisher confirms tidak menutup celah sebelum publikasi; pengembangan lanjutan memerlukan transactional outbox dan consumer idempotent.

BAB V
PENUTUP

5.1 Penerapan pada Proyek Marketplace UMKM

Pemberitahuan pesanan baru dijadikan asinkron. Order Service menerbitkan order.created dan Notification Service membacanya melalui notification-queue; audit dapat memakai antrean terpisah. Event memuat ID pesanan, snapshot produk dan harga, jumlah, total, status, serta waktu. Notifikasi dapat menyusul tanpa menahan checkout, sedangkan reservasi stok tetap sinkron karena hasilnya menentukan penerimaan pesanan.

5.2 Refleksi

Jika RabbitMQ mati saat order dibuat, publisher mencatat bahwa event tidak terkirim, sementara order tetap tersimpan dan klien tetap menerima HTTP 201. Hal itu dapat membuat data pesanan dan notifikasi tidak konsisten, sehingga pengembangan selanjutnya memerlukan penyimpanan order bersama outbox dalam satu transaksi serta relay yang mencoba ulang. Consumer yang idempotent harus memakai messageId unik di penyimpanan tahan restart dan mencatat ID beserta side effect database secara atomik sebelum ack, bukan hanya menggunakan Set dalam memori. Untuk email eksternal, diperlukan outbox pengiriman dan dukungan idempotency key dari penyedia karena transaksi database sendiri tidak menjamin email terkirim tepat sekali. Panggilan sinkron tetap lebih tepat untuk reservasi stok dan pengambilan harga saat checkout karena Order Service harus mengetahui hasilnya sebelum menerima pesanan, sedangkan notifikasi dapat diproses kemudian.

5.3 Kesimpulan dan Batasan

Bukti manual menunjukkan tiga pesan tertahan saat consumer mati, pemrosesan setelah pulih, dan email dua kali akibat crash sebelum ack. Suite otomatis melengkapi verifikasi dengan 75 test dan delapan skenario integrasi yang lulus. Stok, order, dan deduplikasi masih in-memory; transaksi outbox dan idempotensi tahan restart belum diimplementasikan. Pengujian Lab 10 berlangsung lokal melalui Docker; deployment Render yang sudah ada merupakan arsip Lab 09.

DAFTAR PUSTAKA

RabbitMQ. Work Queues Tutorial JavaScript.

RabbitMQ. Publish Subscribe Tutorial JavaScript.

amqplib. Channel API Reference.

Subandoro, Philipus Suryo. Panduan Praktikum Web Service Lab 10 Komunikasi Antar Service via Message Queue. Bahan tugas praktikum.

LAMPIRAN SUMBER DAN REPRODUKSI

Repository pengumpulan tetap menggunakan repository Lab 09 dengan branch lab10-message-queue. Tautan berikut membuka versi Lab 10 beserta sumber laporan dan bukti:

Repository Lab 10 Marketplace UMKM.

Bukti manual utama berada pada docs/evidence/lab10/manual/, terdiri atas tujuh screenshot asli, observations.json sebagai transkripsi, dan SHA256SUMS.txt untuk pemeriksaan keutuhan. Gambar dan logo pada PDF mempertahankan warna aslinya, sedangkan teks, judul, dan tabel tetap hitam putih. Berkas screenshot asli di repository tidak diubah.

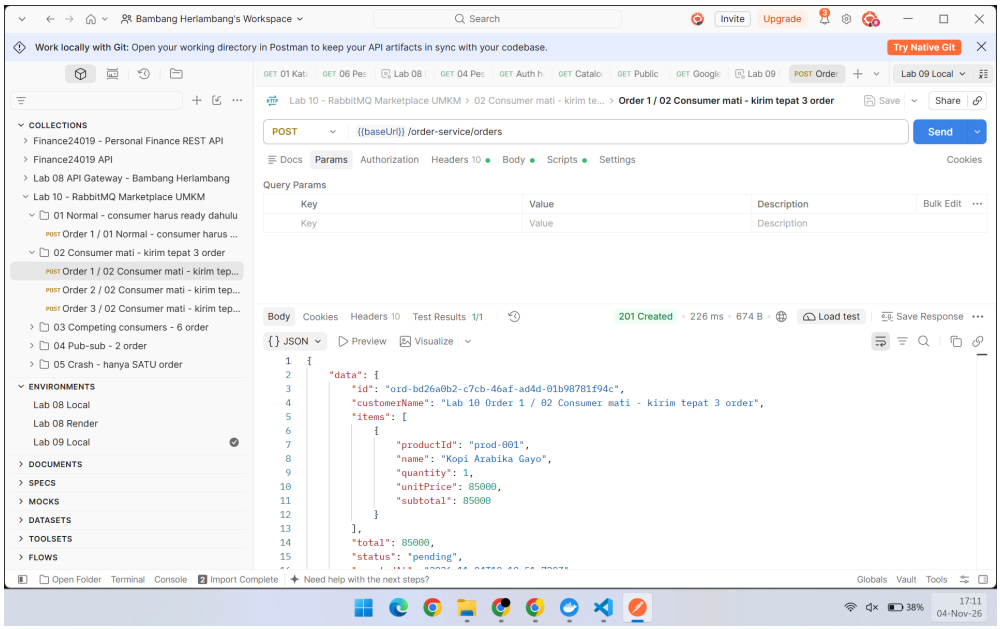
Bukti tambahan pada docs/evidence/lab10/terminal/ memuat screenshot VS Code atas tampilan ringkas log aktual competing consumers dan pub/sub, file JSONL asli, serta hasil HTTP pengujian pada virtual host terpisah. Waktu dan ID run tambahan ini tidak digabung dengan run manual sebelumnya.

Bukti otomatis pelengkap terdapat pada docs/evidence/lab10/results.json, execution.txt, tests.txt, dan log tiap proses. Dokumen lengkap serta prosedur manual tersedia pada docs/lab10/README.md; collection Postman berada pada docs/postman/lab10.postman_collection.json.

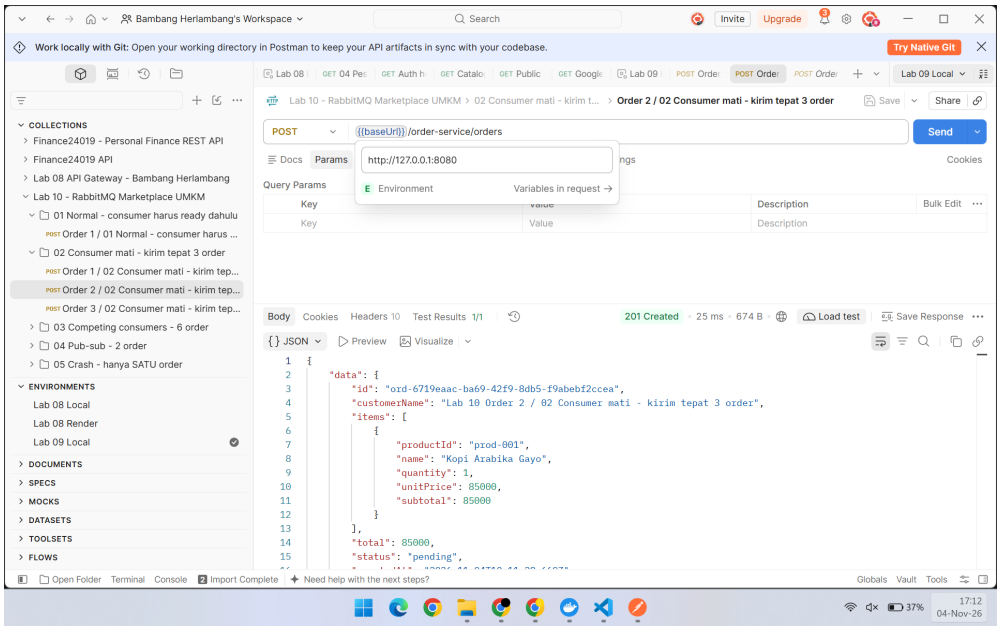
Untuk membangun ulang laporan, jalankan python scripts/build-lab10-report.py menggunakan Python yang memiliki reportlab dan Pillow. Builder membaca bukti yang sudah tersimpan, tidak menjalankan ulang pengujian dan tidak mengubah antrean yang sedang digunakan.

LAMPIRAN BUKTI REQUEST POSTMAN

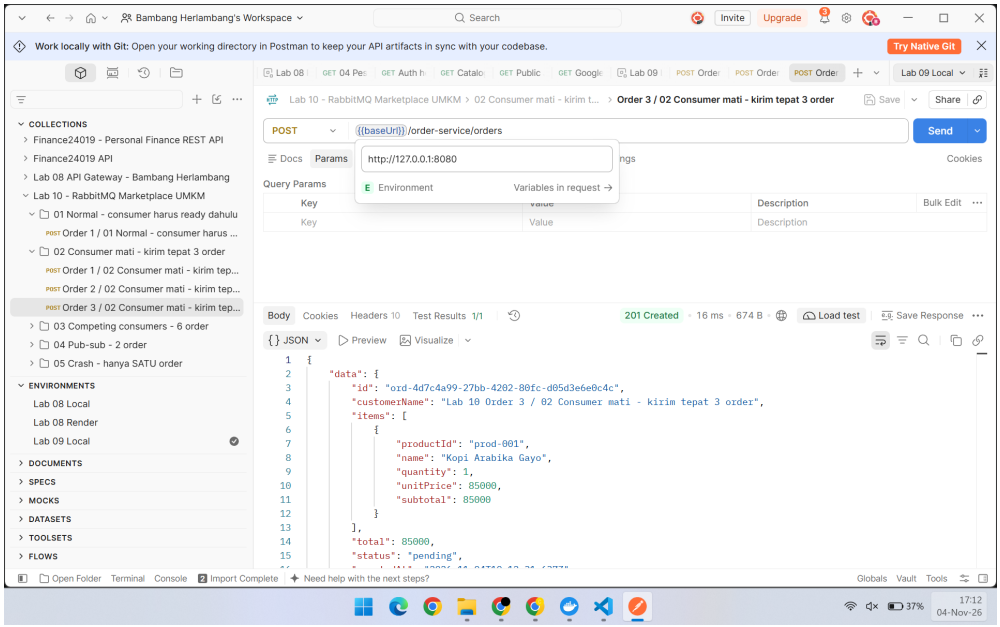
Screenshot Postman berikut adalah lampiran asli pengguna. Masing-masing menunjukkan HTTP 201 Created dan orderId yang dapat dicocokkan dengan log pemulihan pada Subbab 4.3.



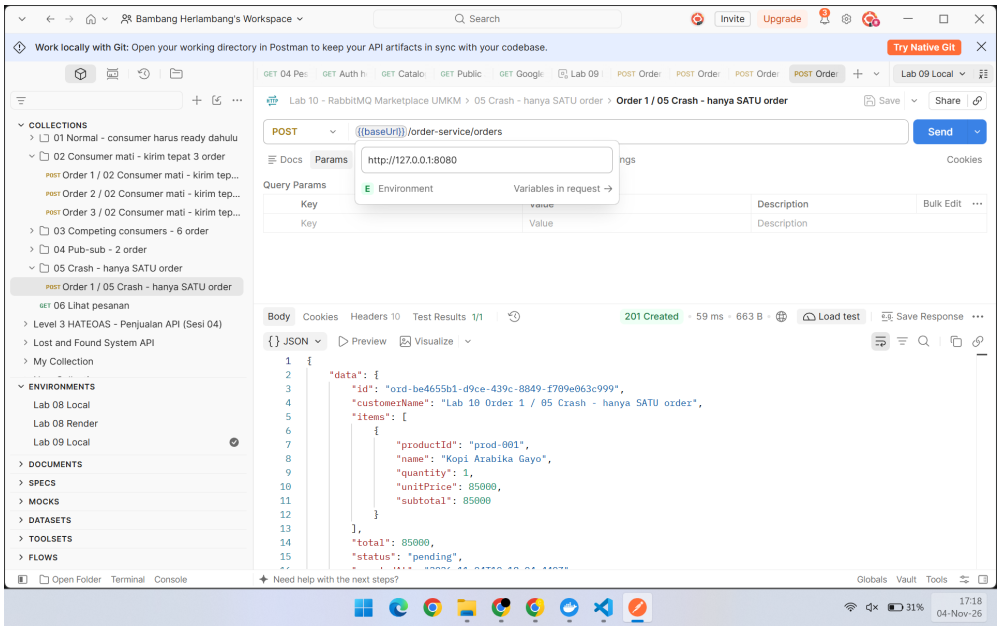
Gambar L.1 Respons Postman pesanan 1 saat consumer mati.



Gambar L.2 Respons Postman pesanan 2 saat consumer mati.



Gambar L.3 Respons Postman pesanan ketiga saat consumer mati.



Gambar L.4 Respons Postman untuk satu pesanan pada uji crash. OrderId yang sama diproses ulang setelah consumer kembali berjalan.